

Análise de Desempenho da Inferência de um Modelo de Linguagem Particionado em Múltiplas GPUs

LUCAS FRAGA BALBINOT, MATHEUS AUGUSTO TREGNAGO, and RAFAEL SILVA DE SOUZA, Universidade Federal do Rio Grande do Sul, Brasil

Estudo do comportamento de Modelos de Linguagem de Grande Porte (*Large Language Models*, ou LLM) em situação de particionamento de dados, utilizando as GPUs nativas aos sistemas da Universidade Federal do Rio Grande do Sul, Campus do Vale.

Additional Key Words and Phrases: Large Language Model, Particionamento de Dados, GPU, Análise de Desempenho

ACM Reference Format:

Lucas Fraga Balbinot, Matheus Augusto Tregnago, and Rafael Silva de Souza. 2026. Análise de Desempenho da Inferência de um Modelo de Linguagem Particionado em Múltiplas GPUs. In *Proceedings of Nome da Conferência (Conferência '26)*. ACM, New York, NY, USA, 4 pages.

1 Introdução

Na era moderna, a prevalência de Inteligências Artificiais no âmbito diário é cada vez mais notável, seja na forma de assistentes virtuais, desistemas de tradução ou mesmo na geração de códigos para programas. Um elemento essencial para sua aplicabilidade cada vez mais abrangente são os *Large Language Models* (LLM), vastas redes neurais capazes de processar bilhões de parâmetros dados por vários usuários eficientemente e tendo tremendo poder computacional para tal tarefa. Contudo, tais atributos também fazem com que LLMs ocupem muito espaço em máquinas, por vezes necessitando que seja particionada em segmentos menores para poder operar.

O propósito deste trabalho é o estudo dos efeitos de particionamento de LLMs por Unidades de Processamento Gráfico (*Graphic Process Unit* ou GPU) e o estudo de como afeta o processo de inferência do modelo, utilizando as GPUs localizadas no Parque Computacional de Alto Desempenho (PCAD)¹ da Universidade Federal do Rio Grande do Sul (UFRGS).

2 Contexto

Inferência é o processo de utilizar um modelo já treinado para gerar texto a partir de uma entrada fornecida pelo usuário. Nos LLMs modernos, baseados na arquitetura Transformer [7], essa geração é autorregressiva: o texto de saída é produzido um *token* por vez, e cada novo token depende de todos os tokens anteriores.

A principal restrição para servir um LLM é a memória da GPU. Os pesos de um modelo com 7 bilhões de parâmetros em precisão *bfloat16* ocupam cerca de 15GB, aos quais se soma o KV cache, que cresce com o número de requisições atendidas. Quando esse total excede a memória de uma única GPU, o modelo precisa ser particionado entre múltiplas GPUs. Neste trabalho, utilizamos duas estratégias de particionamento: Tensor Parallelism e Pipeline Parallelism.

No *Tensor Parallelism* (TP) [4], as matrizes de pesos de cada camada são fatiadas entre as GPUs, de modo que todas as GPUs cooperam no cálculo de todas as camadas. Cada GPU computa um resultado parcial, e os resultados precisam ser

¹<https://pcad.inf.ufrgs.br/>

Authors' Contact Information: Lucas Fraga Balbinot; Matheus Augusto Tregnago; Rafael Silva de Souza, Universidade Federal do Rio Grande do Sul, Porto Alegre, Rio Grande do Sul, Brasil.

2026. Manuscript submitted to ACM

combinados por operações *AllReduce* a cada camada do modelo, em cada passo de geração. A comunicação é, portanto, frequente e está no caminho crítico da inferência.

No *Pipeline Parallelism* (PP) [1], o modelo é dividido em estágios sequenciais de camadas contíguas, cada estágio atribuído a uma GPU. As requisições atravessam os estágios, onde a saída de um estágio é enviada ponto a ponto apenas para o estágio seguinte. O volume de comunicação é muito menor que no TP, limitado às fronteiras entre estágios. Porém, o problema está na ociosidade de computação, visto que apenas um estágio trabalha por vez.

2.1 Pilha de software para inferência distribuída

Na prática, a inferência distribuída é realizada por uma *serving engine* que gerencia o particionamento, o lote dinâmico de requisições e a exposição de uma API de geração. Este trabalho utiliza o vLLM [2], que implementa TP e PP. Em execuções multi-nó, o vLLM utiliza o Ray [3] para coordenar os processos entre as máquinas.

3 Trabalhos Relacionados

O objetivo desse trabalho vai ao encontro de dois estudos recentes. Xu et al. [8] caracterizam os padrões de comunicação da inferência distribuída servida pelo vLLM, combinando *profiling* e modelos analíticos, e concluem que o TP impõe grande volume de comunicação, mas entrega melhor latência para sequências curtas, enquanto o PP minimiza a transferência de dados ao custo de maior latência total. Topcu et al. [6] analisam estratégias de paralelização intra-nó para modelos densos de maior porte, mostrando que o TP favorece objetivos de latência e o PP aplicações orientadas a *throughput*.

Este trabalho revisita a comparação entre TP e PP em um contexto ausente nesses estudos: GPUs de consumo (RTX 4090 e RTX 4070) interligadas por Ethernet Gigabit, em que o custo relativo da comunicação é ordens de grandeza maior. Verificar se as conclusões obtidas sobre interconexões rápidas se sustentam nesse cenário de recursos restritos é a principal contribuição desta análise.

4 Metodologia

A abordagem deste trabalho é experimental, onde o mesmo modelo é servido sob diferentes configurações de particionamento e submetido a cargas controladas de requisições, enquanto o comportamento do sistema é registrado em nível de aplicação, GPU e rede.

O modelo servido em todos os experimentos é o Qwen2.5-7B-Instruct [5], um modelo denso com 7,61 bilhões de parâmetros e 28 camadas. Os pesos tem um tamanho de 15GB e não cabem na GPU de 12GB disponível no cluster, o que torna o particionamento obrigatório nesse hardware, mas cabem na GPU de 24GB, preservando uma configuração de GPU única como *baseline*.

4.1 Métricas avaliadas

No nível da aplicação adotamos as métricas usuais de inferência de LLMs: o tempo até o primeiro token (*Time to First Token*, TTFT); a latência média entre tokens consecutivos (*Inter-Token Latency*, ITL); a latência total da requisição; e as vazões de tokens gerados (tokens/s) e de requisições atendidas (req/s).

No nível da GPU, cada nó registra via `nvidia-smi`, a utilização do processador, a memória ocupada, a potência e a temperatura. No nível da rede, a banda ponto a ponto entre os nós é medida com `iperf3` e a utilização da interface durante a carga é registrada com `sar`. Por fim, traços do Nsight Systems permitem decompor o tempo de GPU entre computação e comunicação.

4.2 Projeto Experimental

O projeto experimental é um fatorial completo sobre os cinco fatores da Tabela 1, do qual são removidas as células inválidas: a configuração sem particionamento exige exatamente 1 GPU, TP e PP exigem pelo menos 2, o modelo não cabe em um único nó poti e a partição tupi foi limitada a 2 nós pela disponibilidade de nós homogêneos. Restam 14 células por tipo de coleta, totalizando 28 execuções.

Table 1. Fatores e níveis do projeto fatorial.

Fator	Níveis
Número de GPUs (1 por nó)	1, 2, 4
Nó do PCAD	tupi, poti
Estratégia de particionamento	nenhuma, TP, PP
Carga (entrada/saída em tokens)	curta 128/128; longa 1024/512
Tipo de coleta	<i>trace</i> ; concorrência

Nas execuções de *trace*, uma única medição de 30 requisições é feita com o servidor instrumentado pelo *nsys* e nas execuções de concorrência, o *nsys* é desligado e o *aiperf* varre os níveis de 1, 2, 4, 8, 16 e 32 requisições simultâneas contra o mesmo servidor. Essas coletas foram separadas em duas execuções diferentes, porque, com diferentes níveis de concorrência, o *trace* do *nsys* acabava ficando muito grande e lotava a partição da máquina no PCAD.

4.3 Ambiente de execução

Os experimentos foram executados no PCAD nas partições descritas na Tabela 2. Cada nó possui exatamente uma GPU, de modo que todo o tráfego de TP e PP atravessa a rede Ethernet de 1 Gbit/s.

Table 2. Especificações técnicas das máquinas utilizadas

Máquina	CPU	RAM	Acelerador	Disco
tupi	Intel(R) Core(TM) i9-14900KF, 3.2 GHz, 32 threads, 24 cores	128 GB DDR5	NVIDIA GeForce RTX4090 24 GB	1.8 TB NVMe
poti	Intel(R) Core(TM) i7-14700KF, 3.4 GHz, 28 threads, 20 cores	96 GB DDR5	NVIDIA GeForce RTX4070 12 GB	119.2 GB NVMe, 1.7 TB SSD

5 Resultados

5.1 Inferência do modelo

5.2 Utilização das GPUs

5.3 Comunicação

6 Limitações do trabalho

7 Conclusão

Considerações finais e propostas de trabalhos futuros.

8 Uso de Inteligência Artificial

Acknowledgments

Agradecemos ao GPPD da UFRGS por poder providenciar o acesso ao PCAD para realizar estes experimentos. Agradecemos ao Professor Dr. Lucas Mello Schnorr pela orientação oferecida no desenvolvimento deste trabalho e seus conselhos.

References

- [1] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. *Advances in Neural Information Processing Systems* 32 (2019).
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*.
- [3] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*.
- [4] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053. <https://arxiv.org/abs/1909.08053>.
- [5] Qwen Team. 2024. Qwen2.5-7B-Instruct Model Hub. <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>. Acessado em: 2026-06-28.
- [6] Burak Topcu, Musa Oguzhan Cim, Poovaiah Palangappa, Meena Arunachalam, and Mahmut Taylan Kandemir. 2026. Parallelization Strategies for Dense LLM Deployment: Navigating Through Application-Specific Tradeoffs and Bottlenecks. arXiv:2603.05692. <https://arxiv.org/abs/2603.05692>.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* 30 (2017).
- [8] Lang Xu, Kaushik Kandadi Suresh, Quentin Anthony, Nawras Alnaasan, and Dhabaleswar K. Panda. 2025. Characterizing Communication Patterns in Distributed Large Language Model Inference. arXiv:2507.14392. <https://arxiv.org/abs/2507.14392>.